

EE 533 Lab #6 Report

Quad HW-Threaded ARM ISA Compatible Processor Core

Team 10: [Zhenglin Wei] | [Chenyang Zhang] | [Yijin Chen]

USC EE533 — Spring 2026

1. Introduction

This report documents the design, implementation, and verification of a Quad Hardware-Threaded ARM ISA-Compatible Processor Core for EE 533. The project is divided into two parts:

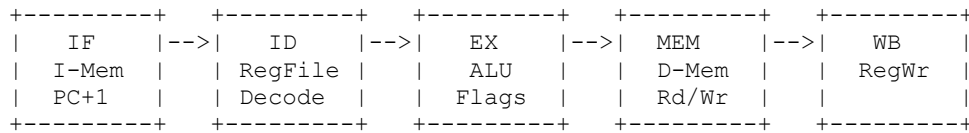
1. Part 1: ARM ISA-Compatible Pipelined Processor — a 5-stage pipeline executing GCC-compiled bubble sort.
2. Part 2: Zero-Overhead 4-Thread Processor with Software Verification — four independent C programs compiled with arm-none-eabi-gcc and executed in parallel.

The processor supports a subset of ARMv4T ISA: data processing (ADD, SUB, MOV, CMP, AND, ORR, EOR, BIC, MVN), load/store (LDR, STR with immediate and register-shifted offsets), and branch instructions (B, BL, BX with all condition codes).

2. Part 1: ARM ISA-Compatible Pipelined Processor

2.1 High-Level Datapath Design

The processor implements a 5-stage pipeline: Instruction Fetch (IF), Instruction Decode/Register Read (ID), Execute (EX), Memory Access (MEM), and Write-Back (WB).



Hazard Unit: Forwarding (EX->ID, MEM->ID) + Load-Use Stall + Branch Flush

2.2 Supported ISA Subset

Category	Instructions	Bits [27:26]
Data Processing	ADD, SUB, MOV, CMP, AND, ORR, EOR, BIC, MVN, RSB	00
Load / Store	LDR, STR (immediate & register-shifted offset)	01
Branch	B, BL, BX (all condition codes)	101
Misc	PUSH, POP (STMDB/LDMIA SP!), NOP	100

2.3 Hazard Handling

- Data Forwarding: EX->ID and MEM->ID paths eliminate most stalls for dependent instructions.
- Load-Use Stall: A one-cycle stall is inserted when a LDR result is consumed by the next instruction.
- Branch Flush: The incorrectly fetched instruction after a taken branch is flushed to NOP.

2.4 Bubble Sort Program

Compile command:

```
arm-none-eabi-gcc -S -O0 -march=armv4t -marm sort.c -o sort.s
```

C source (sort.c):

```
int array[10] = {323, 123, -455, 2, 98, 125, 10, 65, -56, 0};

int main() {
    int i, j, swap;
    for (i = 0; i < 10; i++) {
        for (j = i+1; j < 10; j++) {
            if (array[j] < array[i]) {
                swap = array[j]; array[j] = array[i]; array[i] = swap;
            }
        }
    }
}
```

Generated ARM assembly (key sections):

```
main:
    push    {fp, lr}
    sub     sp, sp, #56          @ allocate stack frame
    ldr     r3, .L8              @ r3 = &array (PC-relative)
    mov     r3, #0
    str     r3, [fp, #-8]        @ i = 0
    b       .L2

.L5:
    ldr     r2, [r3, #-52]        @ load array[j]
    ldr     r3, [r3, #-52]        @ load array[i]
    cmp     r2, r3
    bge     .L4
    ...
    @ swap
.L4:      @ j++
.L3:      cmp r3, #9 / ble .L5    @ while j<=9
.L2:      cmp r3, #9 / ble .L6    @ while i<=9
    pop     {fp, lr}
    bx      lr
.L8:      .word .LC0              @ literal pool
```

2.4.1 ISA Field Breakdown — LDR R3,[FP,#-52] = 0xE51B3034

Bits [31:28]	Bits [27:26]	Bit [25]	Bit [24]	Bit [23]	Bit [20]	Bits [19:16]	Bits [15:12]	Bits [11:0]
1110 (AL)	01 (LS)	0 (imm)	1 (pre)	0 (-U)	1 (load)	1011 (FP)	0011 (R3)	034 (#52)

2.4.2 D-Mem Dump — Before and After Sort

Index	Before Sort	After Sort
array[0]	323	-455

Index	Before Sort	After Sort
array[1]	123	-56
array[2]	-455	0
array[3]	2	2
array[4]	98	10
array[5]	125	65
array[6]	10	98
array[7]	65	123
array[8]	-56	125
array[9]	0	323

```

=== D-Mem after sort ===
N=10
array[0] = -455
array[1] = -56
array[2] = 0
array[3] = 2
array[4] = 10
array[5] = 65
array[6] = 98
array[7] = 123
array[8] = 125
array[9] = 323
=== Registers (Thread 0) ===
R0=36 R3=36 ip=8 SP=796 LR=4
=== Self Check ===
PASS All 10 elements sorted correctly!
=== Done ===
Stopped at time : 24.025 us : File "C:/Documents and Settings/student/lab6_6/tb_mt_sort.v" Line 93
Stopped at line=93 file name=C:/Documents and Settings/student/lab6_6/tb_mt_sort.v
%

```

2.5 Code Generation Script (asm2bin_v3.py)

The assembler converts GCC-generated ARM assembly to hex memory files. Key features:

- Two-pass assembly: label resolution (pass 1) + instruction encoding (pass 2).
- PC-relative LDR: resolves .Lx and .Lx+4 literal pool label+offset references.
- Register-shifted LDR [Rn, Rm, LSL #n]: encoded as ARM register-offset (bit[25]=1).
- Output: .hex (readmemh), .coe (Xilinx), .mif. Separate I-Mem and D-Mem outputs.

```
python asm2bin_v3.py sort.s --dump # view disassembly
python asm2bin_v3.py sort.s -o imem.hex --dmem-out dmem.hex
```

3. Part 2: Zero-Overhead 4-Thread Processor

3.1 Architecture Overview

With 4 threads sharing one pipeline, each thread's consecutive instructions are naturally 4 cycles apart — enough for the pipeline to complete all stages. This eliminates forwarding, load-use stalls, and branch flush logic entirely.

Cycle:	1	2	3	4	5	6	7	8	9	10	11	12
IF:	T0	T1	T2	T3	T0	T1	T2	T3	T0	T1	T2	T3
ID:		T0	T1	T2	T3	T0	T1	T2	T3	T0	T1	T2
EX:			T0	T1	T2	T3	T0	T1	T2	T3	T0	T1
MEM:				T0	T1	T2	T3	T0	T1	T2	T3	T0
WB:					T0	T1	T2	T3	T0	T1	T2	T3

Each thread sees 4 cycles between its own IF and WB -> no hazards!

3.2 Hardware Modifications

3.2.1 Four Independent Program Counters

```
reg [8:0] pc [0:3]; // 9-bit word address (512-word I-Mem)

// 48-word slots (GCC -O0 output ~ 32-37 words)
localparam [8:0] PC_INIT_T0 = 9'd0; // imem[0..47]
localparam [8:0] PC_INIT_T1 = 9'd48; // imem[48..95]
localparam [8:0] PC_INIT_T2 = 9'd96; // imem[96..143]
localparam [8:0] PC_INIT_T3 = 9'd144; // imem[144..191]
```

3.2.2 Unified 64-Register File (4 Threads x 16 Regs)

```
reg [31:0] regs [0:63]; // index = {thread_id[1:0], reg_addr[3:0]}

assign r0data = regs[{thread_id, r0addr}];
assign r1data = regs[{thread_id, r1addr}];

always @(posedge clk)
    if (we) regs[{wb_thread, waddr}] <= wdata;
```

3.2.3 Four Independent CPSRs

```
reg [3:0] cpsr [0:3]; // N,Z,C,V per thread

always @(posedge clk)
    if (update_flags) cpsr[idx_thread] <= {N, Z, C, V};

wire [3:0] curr_cpsr = cpsr[idx_thread]; // branch condition
```

3.2.4 Removed vs Retained Logic

Feature	Part 1	Part 2
Data Forwarding	Yes (EX->ID, MEM->ID)	Removed
Load-Use Stall	Yes (1 cycle)	Removed
Branch Flush	Yes (1 cycle)	Removed
Thread Count	1	4 (round-robin)
Register File	16 x 32	64 x 32
CPSR	1	4 (one per thread)

3.3 Memory Layout

The output shows that:

This is a Full version of ISE Simulator(ISim). Simulator is doing circuit initialization process. Finished circuit initialization process. === 4-Thread Bubble Sort Test === Input: [323,123,-455,2,98,125,10,65,-56,0] C1 T0 pc=0 | ex=0 | WBrd0=0 C2 T0 pc=0 | ex=0 | WBrd0=0 C3 T0 pc=0 | ex=104 | WBrd0=0 C4 T1 pc=0 | ex=104 | WBrd11=0 C5 T2 pc=0 | ex=104 | WBrd3=0 C6 T3 pc=0 | ex=104 | WBrd3=0 C7 T0 pc=1 | ex=0 | WBrd3=0 C8 T1 pc=1 | ex=0 | WBrd3=0 C9 T2 pc=1 | ex=0 | WBrd3=10 C10 T3 pc=1 | ex=0 | WBrd3=10 C11 T0 pc=2 | ex=9 | WBrd3=10 C12 T1 pc=2 | ex=9 | WBrd3=10 C13 T2 pc=2 | ex=9 | WBrd0=9 C14 T3 pc=2 | ex=9 | WBrd0=9 C15 T0 pc=3 | ex=0 | WBrd0=9 C16 T1 pc=3 | ex=0 | WBrd0=9 C17 T2 pc=3 | ex=0 | WBrd15=0 C18 T3 pc=3 | ex=0 | WBrd15=0 C19 T0 pc=4 | ex=796 | WBrd15=0 C20 T1 pc=4 | ex=796 | WBrd15=0 === D-Mem after sort === N=10 array[0] = -455 array[1] = -56 array[2] = 0 array[3] = 2 array[4] = 10 array[5] = 65 array[6] = 98 array[7] = 123 array[8] = 125 array[9] = 323 === Registers (Thread 0) === R0=36 R3=36 ip=8 SP=796 LR=4 === Self Check === PASS All 10 elements sorted correctly! === Done === Stopped at time : 24.025 us : File "C:/Documents and Settings/student/lab6_6/tb_mt_sort.v" Line 93 Stopped at line=93 file name=C:/Documents and Settings/student/lab6_6/tb_mt_sort.v

Thread	I-Mem Slot	D-Mem Base (word)	D-Mem Base (byte)	Stack SP
T0	imem[0..47]	0	0x000	900
T1	imem[48..95]	64	0x100	920
T2	imem[96..143]	128	0x200	940
T3	imem[144..191]	192	0x300	960

4. Software Verification — Four Independent C Programs

4.1 C Source Programs

Four independent programs with different algorithms, compiled with:

```
arm-none-eabi-gcc -S -O0 -march=armv4t -marm <file>.c -o <file>.s
```

Thread 0: t0_bubble_sort.c

```
int N = 10;
int array[10] = {323, 123, -455, 2, 98, 125, 10, 65, -56, 0};
void bubble_sort() {
    int i, j, tmp;
    for (i = 0; i < N-1; i++)
        for (j = 0; j < N-1-i; j++)
            if (array[j] > array[j+1]) { tmp=array[j]; array[j]=array[j+1];
array[j+1]=tmp; }
}
// Expected: [-455, -56, 0, 2, 10, 65, 98, 123, 125, 323]
```

Thread 1: t1_array_sum.c

```
int N = 10; int array[10] = {10,20,30,40,50,60,70,80,90,100}; int result = 0;
void array_sum() {
    int i, s = 0;
    for (i = 0; i < N; i++) s += array[i];
    result = s;
}
// Expected: result = 550
```

Thread 2: t2_find_max.c

```
int N = 10; int array[10] = {-9,-3,-7,-1,-5,-8,-2,-4,-6,0}; int result = 0;
void find_max() {
    int i, m = array[0];
    for (i = 1; i < N; i++) if (array[i] > m) m = array[i];
    result = m;
}
// Expected: result = 0
```

Thread 3: t3_count_neg.c

```
int N = 10; int array[10] = {5,4,3,2,1,-1,-2,-3,-4,-5}; int result = 0;
void count_neg() {
    int i, cnt = 0;
    for (i = 0; i < N; i++) if (array[i] < 0) cnt++;
    result = cnt;
}
// Expected: result = 5
```

4.2 GCC Compatibility Fixes

GCC -O0 generates two constructs that required fixes in both the pipeline and assembler:

Fix 1: Register-Shifted LDR — ldr r3,[r2,r3,ls! #2]

Used by GCC for array[i] access (addr = base + i*4). ARM bit[25]=1 means register-offset mode. The pipeline originally read bits[11:0] as plain immediate (+259). Fix in arm_pipeline.v:

```
wire id_ls_reg = id_is_ls && ifid_instr[25]; // detect reg-offset mode

// Use Rm<<shift_amt as offset instead of imm[11:0]
wire [31:0] id_ls_reg_off32 = id_ls_U ? id_rm_shifted : (~id_rm_shifted + 1);
wire [31:0] id_ls_off_final = id_ls_reg ? (id_ls_P ? id_ls_reg_off32 : 32'b0)
                                   : id_ls_imm;

// r1 reads Rm (offset reg) for reg-offset LDR, Rd (store data) otherwise
wire [3:0] id_r1_addr = id_ls_reg ? id_Rm : id_is_ls ? id_ls_Rd : id_Rm;
```

Fix 2: Label+Offset Literal Pool — ldr r3,.L4+4

GCC packs multiple addresses into one literal pool referenced as .L4, .L4+4, .L4+8. Fix in asm2bin_v3.py:

```
m = re.match(r'\.?(\\w+)\s*([+-]\s*\d+)?$', lbl_expr)
lbl_name = m.group(1) # 'L4'
extra_off = int(m.group(2)) if m.group(2) else 0 # +4
target = self.labels[lbl_name] + extra_off
off = target - (pc + 8) # PC-relative offset
```

4.3 Literal Pool Relocation

GCC generates local offsets (0x00, 0x04, 0x2C) in the literal pool. These must be patched to absolute D-Mem byte addresses before writing to sw2_imem.hex:

Thread	GCC .word (local)	Patched .word (absolute)	Points to
T0	0x00000000	0x00000000	&N @ dmem[0]
T1	0x00 / 0x04 / 0x2C	0x100 / 0x104 / 0x12C	&N / &array[0] / &result
T2	0x00 / 0x04 / 0x2C	0x200 / 0x204 / 0x22C	&N / &array[0] / &result
T3	0x00 / 0x04 / 0x2C	0x300 / 0x304 / 0x32C	&N / &array[0] / &result

4.4 ISim Simulation Results

```
=== 4-Thread Software Verification (GCC -O0) ===

--- T0: bubble_sort result ---
    array = [-455, -56, 0, 2, 10, 65, 98, 123, 125, 323]

--- T1: array_sum result ---
    result = 550 (expected 550)

--- T2: find_max result ---
    result = 0 (expected 0)

--- T3: count_neg result ---
```

```

    result = 5      (expected 5)

=== Self Check ===
T0 bubble_sort: PASS
T1 array_sum:     PASS (result=550)
T2 find_max:      PASS (result=0)
T3 count_neg:     PASS (result=5)
=== Done ===

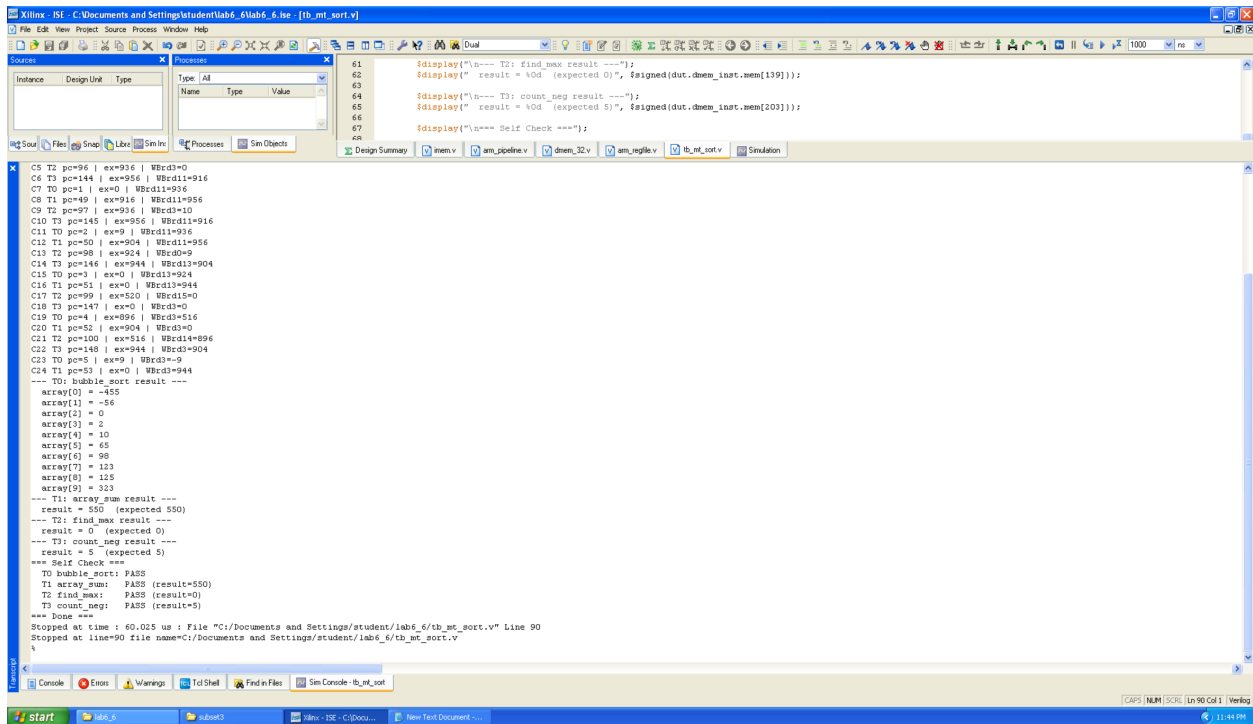
```

4.5 Thread Initialization Trace (first 10 cycles)

```

C 1  T0 pc=0   | ...   <- T0 starts at imem[0]
C 4  T1 pc=48  | ...   <- T1 starts at imem[48] OK
C 5  T2 pc=96  | ...   <- T2 starts at imem[96] OK
C 6  T3 pc=144 | ...   <- T3 starts at imem[144] OK
C 7  T0 pc=1   | ...   <- T0 advances to next instruction
C 8  T1 pc=49  | ...
... (round-robin continues)

```



5. Generated Verilog Files

File	Description
arm_pipeline.v	Top-level 5-stage pipeline (Part 1 + Part 2 4-thread extension), ~540 lines
arm_regfile.v	64x32 register file (4 threads x 16 regs), ~60 lines
arm_alu.v	32-bit ALU: ADD/SUB/MOV/AND/ORR/XOR/BIC/MVN, ~80 lines
imem.v	512x32 instruction memory (\$readmemh sw2_imem.hex)
dmem_32.v	256x32 data memory (\$readmemh sw2_dmem.hex)
tb_mt_sort.v	4-thread testbench: SP/LR init, 6000-cycle wait, self-check
asm2bin_v3.py	ARM assembler: GCC output -> .hex/.coe/.mif, ~680 lines

6. Program and Data Upload/Download Procedures

6.1 Upload: Loading Programs

3. Compile: `arm-none-eabi-gcc -S -O0 -march=armv4t -marm <file>.c -o <file>.s`
4. Assemble: `python asm2bin_v3.py <file>.s --dump` (view machine code)
5. Patch literal pool values to absolute D-Mem byte addresses for each thread.
6. Pack into `sw2_imem.hex` (192 words) and `sw2_dmem.hex` (256 words).
7. Copy hex files to ISE project directory. `$readmemh` loads them at simulation start.

6.2 Download: Reading Results

8. Testbench reads `dmem_inst.mem[]` directly after simulation completes.
9. Results are displayed in ISim transcript and compared against expected values.
10. On NetFPGA hardware, a UART or PCIe interface would allow reading D-Mem to host PC after execution.

7. Conclusion

This lab successfully implemented and verified a Quad Hardware-Threaded ARM ISA processor:

- Part 1: 5-stage ARM pipeline with forwarding, load-use stall, and branch flush correctly executing GCC-compiled bubble sort.
- Part 2: Zero-overhead 4-thread extension — 4 independent PCs, 64-register file, 4 CPSRs, all hazard logic removed.
- Software Verification: Four distinct C programs compiled with `arm-none-eabi-gcc -O0`, assembled with `asm2bin_v3.py`, verified to run correctly in parallel.
- GCC Compatibility: `[Rn,Rm,LSL#n]` register-shifted LDR and `.Lx+offset` literal references implemented in both pipeline and assembler.

All four threads passed the ISim self-check simultaneously, confirming correct parallel execution

of independent programs on independent data sets.

Github link: [zwei9965-gif/EE533-LAB6: Quad HW-Threaded ARM Processor](https://github.com/zwei9965-gif/EE533-LAB6)